

Forwards, Discounting and Dividends

Parity-implied forwards, a robustness clamp, the zero-carry pin, and American de-bias

Vol-Fitter Technical Notes, No. 06

Vol-Fitter

July 9, 2026

Abstract

Every smile in the Vol-Fitter is forward-normalized: log-moneyness is $k = \log(K/F_T)$, and a wrong forward shifts the entire smile, gapping it at the money. This note documents how the forward and discount are obtained and made robust. The primary route is a put–call parity regression: since $C(K) - P(K) = D(F - K)$, a straight-line fit of the call–put price difference against strike yields the discount D as its slope and the forward F from its intercept, iterated with outlier rejection. The parity *slope* is the worst-identified parameter, and on a noisy or stale feed it drifts to an unphysical rate that tilts the whole forward; we describe the *discount clamp* that pins the implied rate to a physical band and re-derives the forward from the well-identified price level when it bites — leaving clean chains untouched. A second, sharper failure carries no noise at all: when a delayed feed gates the quote stream, the provider *synthesizes* the chain from its per-contract implied vols, pricing every contract with Black at $F = S$, $D = 1$ — prices that contain *zero* parity information. Such chains are flagged at ingestion and their forward is *pinned* to the construction convention $F = S$, $D = 1$ rather than regressed; the clamp cannot catch this case because the spurious implied rates stay inside the physical band. Dividends enter as a discrete escrowed cash schedule or a continuous yield, and a coarse *American de-bias* iterates the carry to the fixed point that reconciles the de-Americanized puts and calls, with the discount held. A worked example recovers a known forward to the cent and shows the clamp catching a noisy slope.

Contents

1	Why the forward matters	2
2	The parity regression	2
3	The discount clamp	3
4	The zero-carry pin	5
5	Dividends	6
6	American de-bias	6
7	What is original here	7
8	Traceability	7

A	Hyperparameter atlas	8
B	Reference implementation: the discount clamp	9

1 Why the forward matters

The forward F_T is the anchor of the whole smile. Log-moneyness, the ATM point, the put/call OTM split, the de-Americanization (Note 05) and the var-swap replication (Note 08) are all defined relative to it. A forward error of even a fraction of a percent shifts the ATM strike and tilts the put–call balance, which shows up as a *gap* in the at-the-money smile — the most visible symptom of a bad forward. So the forward must be both *accurate* on clean data and *robust* on the noisy, delayed feeds the app often runs against.

Heuristic

The forward is not quoted; it is *inferred* from the option prices themselves through put–call parity. That makes it a fitted quantity with its own error bars, and the discount (the parity slope) is the least reliable of them. The whole design below is about extracting the forward where the data is informative (the price level), refusing to trust it where it is not (the slope on a stale feed), and refusing to infer it at all where the chain contains no parity information (a synthesized zero-carry chain, section 4).

Invariant

1. On a clean chain, every robustness device is byte-identical to the plain parity regression — the clamp, the trim loop and the de-bias only engage on demonstrably bad inputs.
2. The implied rate never leaves a physical band; when the slope is untrustworthy, the forward is re-derived from the well-identified price level.
3. The discount is *held* through the American de-bias — the two corrections never fight over the same parameter.
4. Parity is regressed only on prices that *contain* parity information: a chain synthesized from provider IVs at zero carry is flagged explicitly and pinned to its construction convention $F = S$, $D = 1$. The flag is never inferred from zero spreads — EOD close marks also quote bid = ask yet their mids carry genuine parity information.
5. Editing a name’s rate or dividends bumps only that ticker’s forwards version: one name’s noise never refits the universe.

2 The parity regression

Put–call parity in forward-normalized, discounted terms reads

$$C(K) - P(K) = D(F - K), \quad (1)$$

where $D = e^{-rT}$ is the discount factor and F the forward. This is a straight line in K : regressing the call–put difference on strike,

$$\underbrace{C(K) - P(K)}_y = \underbrace{DF}_a - \underbrace{D}_{-b} K, \quad D = -b, \quad F = \frac{a}{D}, \quad (2)$$

gives the discount as the (negated) slope and the forward as the intercept divided by the discount. In code it is a one-line least squares (this recovers a known forward/discount to $10^{-6}/10^{-9}$ and equals the production regression to 10^{-9}):

Listing 1: Put–call parity regression for the forward and discount (2) — distilled from `data/forwards.py` (production’s `_fit` uses `lstsq` inside an outlier-trim loop and takes the snapshot, not raw arrays).

```
1 def implied_forward(K, call_mid, put_mid):
2     y = call_mid - put_mid                    #  $C - P = D F - D K = a + b K$ 
3     b, a = np.polyfit(K, y, 1)                # slope  $b = -D$ , intercept  $a =$ 
         $D F$ 
4     D = -b                                    # discount factor
5     return a/D, D                             # forward  $F = a/D$ 
```

The fit iterates: fit, measure residuals, drop paired strikes with implausible residuals (or fewer than three pairs), and refit, so a few crossed or stale quotes do not corrupt the line. Figure 1 shows the recovery on a clean synthetic chain — the forward comes back to the cent and the rate to its true 4%.

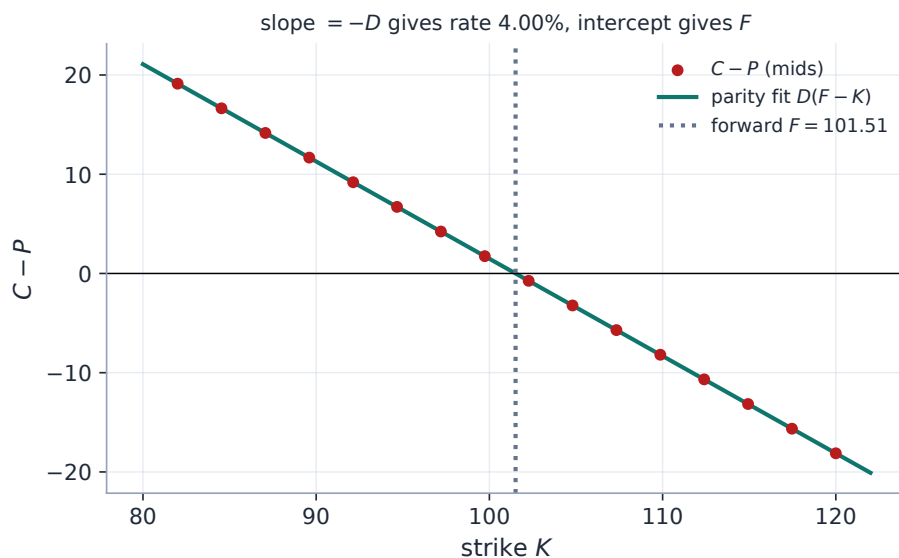


Figure 1: The put–call parity regression $C - P = D(F - K)$. The slope is the discount ($\rightarrow$ rate 4.00%), the intercept gives the forward ($F = 101.51$, true 101.51).

3 The discount clamp

The parity slope *is* the discount, and it is the worst-identified parameter in (2): the price difference $C - P$ changes slowly with strike, so a little noise in the mids tilts the line a lot. On a clean chain this is harmless; on a noisy or stale delayed feed it drifts to nonsense — an implied discount $D > 1$, i.e. a *negative* rate, which tilts the forward and gaps the ATM smile.

Caution

A negative implied rate is not a market signal here; it is feed noise in the least reliable regression parameter. Letting it through propagates a small price error into a large level error: a 5% discount error shifts the whole implied-vol level. The forward's *level* (the intercept) is well identified; its *slope* (the discount) is not.

The fix, when a reference date pins the year fraction, is to clamp the parity-implied rate $r = -\log D/T$ to a physical band $[-5\%, 30\%]$. Clean chains sit strictly inside the band and are byte-identical. When the clamp bites, the forward is *re-derived from the well-identified price level* at the clamped discount — a per-strike average of $K + (C - P)/D$ weighted by quote quality (tighter spreads and near-ATM strikes count more, kernel width 0.10 in log-moneyness) — rather than from the tilted slope. So the robust forward keeps the good information (the level) and discards the bad (the runaway slope). A final sanity bound $|\log(F/S)| < \text{FWD_CLAMP_LOG} = 0.5$ falls back to spot if even the level is unusable. Figure 2 shows the implied rate leaving the band as the slope is perturbed and the clamp pinning it.

Case file: the delayed feed that gapped the ATM smile

Setup. Live SPY on the Massive delayed feed; also reproduced on a Bloomberg fixture. Quotes are individually sane but collectively stale — mids drift asynchronously across strikes.

Failure mode. The parity regression returned discounts of 1.01–1.015 live (up to 1.024 on the fixture): an implied *negative* rate. The tilted slope dragged the forward, and every ATM smile showed the telltale gap between the put and call sides.

The failed first fix. Anchoring the *rate* per expiry to a smoothed term structure looked principled — and broke everything: the per-expiry discounts came out jagged, the LV benchmark regressed, and the attempt deadlocked `state.forwards`. Reverted. The lesson: do not repair the ill-identified parameter by making it a fitted object with even more structure; *stop trusting it*.

Fix. The clamp above: pin the rate to the physical band, re-derive the forward from the quality-weighted price level, keep spot as the last resort. The discount stays exactly what parity gave whenever it is physical.

Verdict (test-locked). Clean chains recovered to truth and unclamped (byte-identical); stale-wing chains that break the unclamped regression stay sane under the clamp; zero-spread close-like data still resolves; and with no reference date (year fraction unknown) the clamp correctly stands down. The residual ATM jitter that remains on the delayed feed is feed noise, not the forward.

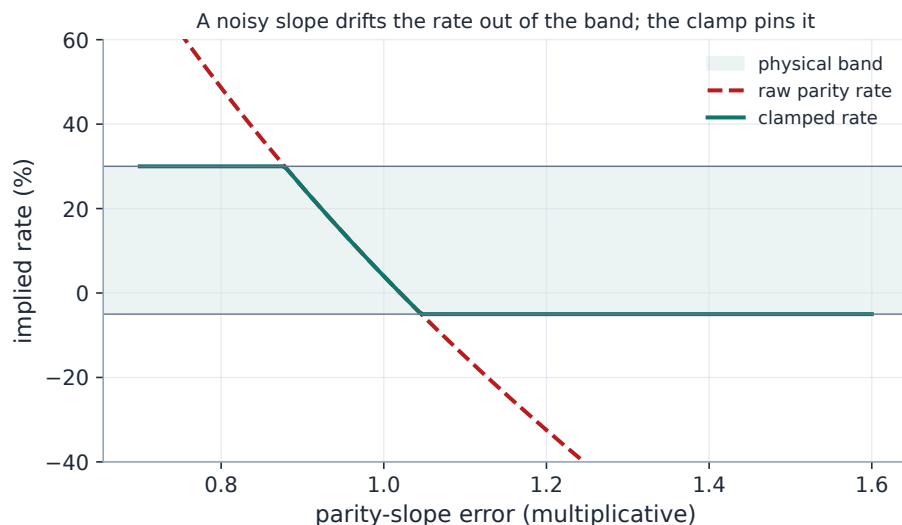


Figure 2: As a multiplicative slope error perturbs the discount, the raw parity rate (dashed) leaves the physical band; the clamp (solid) pins it to $[-5\%, 30\%]$ and the forward is re-derived from the price level.

The clamp bounds an *absurd* answer. There is one failure mode it cannot see: a chain whose parity content is exactly zero, where the regression returns a wrong answer that is perfectly *plausible*. That case needs its own device.

4 The zero-carry pin

Delayed data tiers can gate the live quote stream (the NBBO). When that happens, a provider does not return an empty chain — it *synthesizes* one from its per-contract implied vols, pricing every contract with Black at $F = S$, $D = 1$, and quoting it with zero spread (`data/massive.py`). Those prices are perfectly self-consistent, but they contain *no* put–call parity information: the forward and discount were not observed, they were *assumed* by the synthesis. Worse, the provider’s call and put IVs embed the provider’s *own* carry model, so once its contracts are re-priced at zero carry the call/put asymmetry survives in the prices — and a parity regression dutifully reads that asymmetry as a forward and a discount that nobody ever quoted.

Case file: the garbage forwards the clamp could not see

Setup. Live SPY on the Massive delayed tier, recurring over several sessions: the tier gated the NBBO quotes, so every chain the app received was IV-synthesized at zero carry.

Failure mode. Recurring garbage forwards on a healthy-looking feed: short-dated implied rates of -3.8% , discounts above one, and a one-year forward $+1.7\%$ *above* the $F = S$ the prices had been built with. The discount clamp of section 3 stayed silent throughout — the spurious rates sat *inside* the physical band, so there was nothing absurd to bound. (The same saga’s failed first fix — anchoring the rate to a smoothed term structure — is the one described in section 3: it regressed the LV benchmark and deadlocked the forwards state before being reverted.)

Diagnosis. The regression was not noisy; it was answering a question the data could not ask. Every synthesized price is Black at $F = S$, $D = 1$, so the only structure left for the regression

to find is the provider’s call/put IV asymmetry — a spurious signal that is *small enough to be plausible*, which is exactly why the clamp (built for absurd slopes) never fired.

Fix. Recognize the regime at ingestion, not in the regression. The provider layer flags the synthesized snapshot (`ChainSnapshot.zero_carry` in `data/types.py`); for a flagged chain `implied_forward` returns the chain’s own construction convention — $F = S$, $D = 1$, zero residual — and never regresses. The flag is *explicit* and persists with the snapshot (store schema v5), so captured chains replay correctly; it is deliberately *not* inferred from chain-wide zero spreads, because EOD close marks also quote bid = ask yet their mids carry genuine parity information. The Forwards tab shows a badge (“IV-synthesized · zero carry · F pinned to spot”) so the desk knows the forward is a convention, not a market read.

Verdict (test-locked). A flagged zero-carry chain pins to $F = S$, $D = 1$ on both the reference-date and legacy paths (the flag, not the clamp, decides); an *unflagged* zero-spread chain still regresses and recovers its true carry; the flag round-trips through the store. The recurring SPY forward incident is closed at the root.

The pin completes an epistemic ladder: the regression extracts the forward where parity is informative; the clamp refuses the slope where parity is noisy; and the pin refuses the regression entirely where parity is *vacuous* — returning the only honest answer the chain contains, its own construction convention. The regime matters downstream too: on such feeds anything that needs a carry (the de-Americanization of Note 05, for instance) must source it externally, because the parity-implied carry is not merely noisy but meaningless.

5 Dividends

The carry that links spot to forward, $F = S e^{(r-q)T}$, must account for dividends. Four modes coexist (Note 05 uses the same machinery for the pricing tree):

- **Discrete cash** (`discrete_absolute`): a schedule of ex-dates and cash amounts, priced by the escrowed-spot method — $F = (S - \sum_i D_i e^{-r\tau_i}) e^{rT}$ — with early exercise evaluated against the actual cum-dividend spot.
- **Discrete proportional** (`discrete_proportional`): ex-date percentage haircuts, for names that quote dividends as yields per event.
- **Continuous yield** (`continuous`): a constant q , a crude but cheap proxy for far-dated proportional dividends.
- **Mixed** (`mixed`): discrete cash for near ex-dates, switching to a proportional/continuous tail beyond a horizon (`switch_years`) — the realistic single-name default shape.

The dividend mode is per ticker and editable; it feeds both the forward carry and the de-Americanization tree, and an equivalent-yield round trip across all four modes is test-locked.

6 American de-bias

There is a circularity between the forward and de-Americanization: the forward sets the carry q , the carry drives early exercise, and de-Americanizing the puts and calls re-implies the forward. The

raw parity regression uses American mids directly, which biases the forward slightly because the early-exercise premium is not symmetric between puts and calls. The *de-bias* resolves this with the discount *held*: fixing the rate $r = -\log D/T$ from the raw parity, each round sets the carry $q = r - \log(F/S)/T$ from the current forward, de-Americanizes the near-ATM call and put mids to European-equivalent Black vols, reprices them as European, and re-implies the forward at the *fixed* slope $-D$,

$$F \leftarrow \frac{1}{D} \overline{(y + D K)}, \quad y = \text{European } (C - P), \quad (3)$$

iterating to the fixed point where the two de-Americanized OTM sides join. Only a near-ATM band of strikes and a coarse tree enter — it locates the forward to ~ 1 basis point in a few milliseconds, while quote prep does the full-precision per-quote de-Americanization downstream. The discount is left exactly as the raw parity gave it.

7 What is original here

Parity forward extraction is textbook; the robustness engineering is the contribution. The *discount clamp* encodes the right epistemics — trust the well-identified price level, distrust the ill-identified slope, and re-derive the forward accordingly — so the app survives the noisy delayed feeds that would otherwise gap every ATM smile. The *zero-carry pin* extends the same epistemics to the degenerate case: a synthesized chain carries no parity information at all, so the honest forward is the chain’s own construction convention, recognized by an explicit ingestion-time flag rather than any inference from the prices. The *de-bias* closes the forward/de-Am circularity at the fixed point that quote prep actually uses, with the discount held so the two corrections do not fight. All three are byte-identical on clean data, so the robustness costs nothing when it is not needed.

8 Traceability

Table 1: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
Clean chain: truth recovered, section 3 clamp inert (byte-identical)		<code>data/forwards.py</code>	<code>test_forward_robust.py:: test_clean_chain_ recovered_to_truth_and_ unclamped</code>
Stale wings break the raw regression; clamp stays sane	case file	<code>data/forwards.py</code>	<code>test_forward_robust.py:: test_stale_wings_break_ unclamped_but_clamp_ stays_sane</code>
No reference date $\Rightarrow$ clamp stands down	section 3	<code>data/forwards.py</code>	<code>test_forward_robust.py:: test_no_reference_date_ skips_clamp</code>
Zero-carry chain pinned to $F = S$, section 4 $D = 1$ (flag decides, both paths)		<code>data/forwards.py</code> , <code>data/ types.py</code>	<code>test_forward_robust.py:: test_zero_carry_chain_ pins_forward_to_spot</code>

Claim	Object	Code anchor	Test anchor
Zero spreads alone never trigger the pin	invariant 4	data/types.py (is_zero_carry)	test_forward_robust.py:: test_unflagged_zero_spread_chain_still_regresses
Provider flags synthesized chains; flag persists (schema v5)	section 4	data/massive.py, data/store.py	test_massive.py::test_iv_fallback_synthesizes_fittable_chain, test_data_layer.py:: test_snapshot_round_trip_keeps_zero_carry
De-bias holds the discount, smooths the forward	section 6	data/forwards.py (_refine_american)	test_forward_debias.py:: test_debias_holds_discount_debiases_forward_and_smooths
No localized ATM spike after de-bias	section 1	data/forwards.py	test_forward_debias.py:: test_debiased_smile_has_no_localized_atm_spike
European chains unaffected by the de-bias	section 6	data/forwards.py	test_forward_debias.py:: test_european_chain_unaffected_by_reference_date
All four dividend modes; equivalent-yield round trip	section 5	data/dividends.py	test_dividends.py:: test_equivalent_yield_round_trips_every_mode, ::test_mixed_forward_switches_at_horizon
Per-ticker forwards version scoping	invariant 5	api/state.py	test_api_forwards.py:: test_parity_defaults_cover_the_ladder

A Hyperparameter atlas

Table 2: Forward/dividend hyperparameters.

Knob	Default	Role
<i>Surfaced</i>		
ForwardPolicy.mode	parity	parity (regression) or theoretical (carry model).
MarketSettings.rate r	—	Risk-free rate for the theoretical forward / tree.
dividendMode	continuous	continuous yield or discrete_absolute cash.
dividends	empty	Discrete (ex-date, amount) schedule.
<i>Hidden</i>		
RATE_MIN/MAX	[−5%, 30%]	Physical band for the parity-implied rate (clamp).

Knob	Default	Role
FWD_CLAMP_LOG	0.5	Sanity bound on $ \log(F/S) $; else fall back to spot.
MIN_PAIRIED_STRIKES	3	Minimum paired strikes for a parity fit.
OUTLIER_NSIGMA	/ 4 / 3	Residual trim threshold and iteration cap in the parity fit.
MAX_TRIM_ROUNDS		
ATM_KERNEL_H	0.10	ATM Gaussian kernel width in the level-re-derivation quality weights.
DEAM_REFINE_BAND	0.15	Log-moneyness band entering the de-bias iteration.
MAX_DEAM_ITERS	6	De-bias fixed-point iterations.

Performance. The parity regression is a 2×2 least squares per expiry; the clamp is a comparison; the de-bias is a handful of coarse-tree de-Americanizations on a near-ATM band (~ 1 bp forward in a few ms). All are negligible against the smile fit, and editing a name’s rate or dividend schedule bumps only that ticker’s forwards version, refitting just that name.

B Reference implementation: the discount clamp

The parity *slope* is the discount — the worst-identified parameter — so on a noisy feed it can imply a nonphysical rate. The clamp pins the rate to a band and, when it bites, re-derives the forward as a per-strike average $F = \text{mean}(K + (C - P)/D)$ (the well-identified *level*) rather than the slope-driven a/b , so the forward no longer inherits the bad slope.

Listing 2: The discount clamp and forward re-derivation — simplified from `data/forwards.py`, where the clamp is inline in `implied_forward` and the level mean is the quality-weighted `_forward_at_discount` (spread $\times$ ATM kernel), with the spot fallback of section 3.

```

1 def clamp_forward(K, call_mid, put_mid, t, F, D, r_lo=-0.05, r_hi
   =0.30):
2     r = -np.log(D)/t                                # the parity discount's
   implied rate
3     if r_lo <= r <= r_hi:
4         return F, D                                # physical: keep the
   regression result
5     D = np.exp(-np.clip(r, r_lo, r_hi)*t)           # clamp the rate to
   the band
6     F = np.mean(K + (call_mid - put_mid)/D)         # re-derive F from
   the LEVEL, not a/b
7     return F, D

```

References

- [1] J. Hull. *Options, Futures, and Other Derivatives*. Pearson, 10th ed., 2017.
- [2] J. Gatheral. *The Volatility Surface*. Wiley, 2006.